Class 9: Deep Network Architectures/Interpretability

IALAB UC

Computer Science Department, PUC

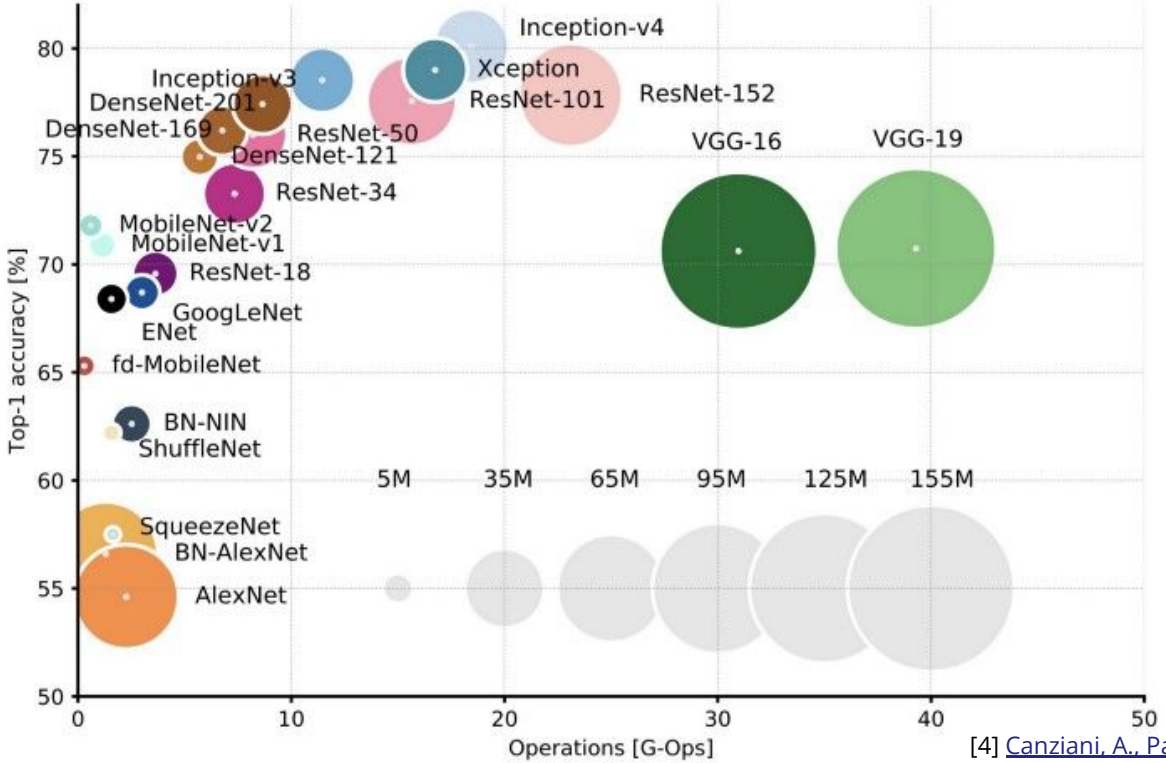
Today’s schedule

- Deep Learning Architectures
  - VGG
  - Inception
  - Resnet

- Interpretability
  - Feature visualization
  - Attribution

Deep Learning Architectures

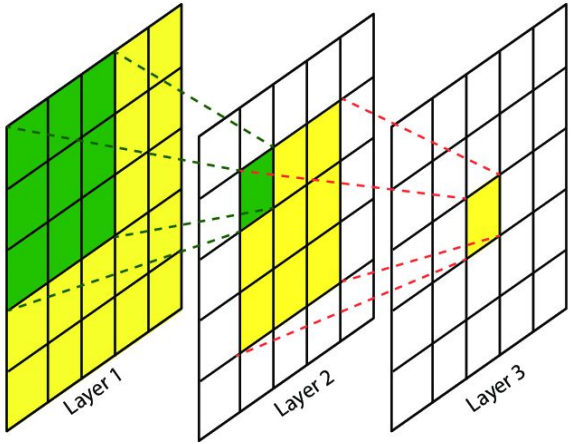
Same dataset, different results



[4] Canziani, A., Paszke, A., & Culurciello, E. (2016)

Preliminaries: Receptive Field

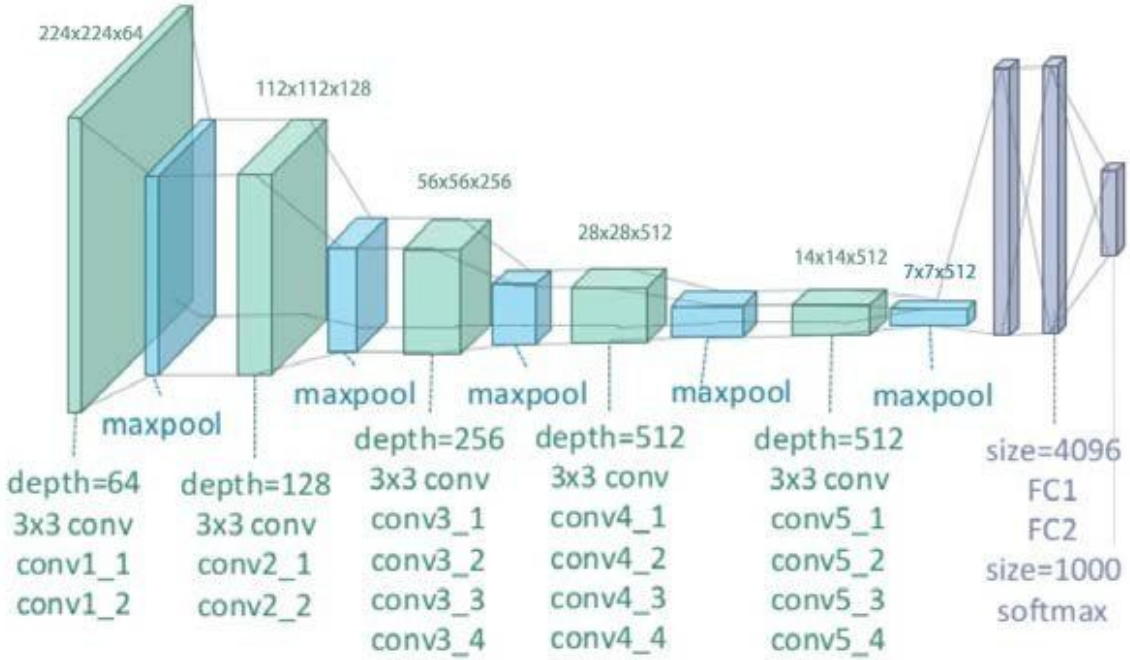
- Effective portion of input that generated a neuron



- Layer 3 neurons are build with 3x3 filters, but have a receptive field of 5x5 with respect to the input

VGG

- Deepest network of its time



[1] Simonyan, K., & Zisserman, A. (2014).

VGG

- Key insight
  - 3x3 filters
    - Smallest filter that captures the notion of up/down, left/right, center
    - Composition of 3x3 filters has same receptive field that larger filters but uses less parameters
      - A single layer with a filter of 5x5 has a receptive field of 5x5 and 25 parameters
      - Two stacked layers with filters of 3x3 have a receptive field of 5x5 and 18 parameters (3x3x2)
      - Less parameters implies less prediction time and less overfitting

VGG: Configurations

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 **LRN**	conv3-64 **conv3-64**	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 **conv3-128**	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 **conv1-256**	conv3-256 conv3-256 **conv3-256**	conv3-256 conv3-256 conv3-256 **conv3-256**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 **conv1-512**	conv3-512 conv3-512 **conv3-512**	conv3-512 conv3-512 conv3-512 **conv3-512**
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: **Number of parameters** (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

Inception

- Multiscale detection

- Reduced number of parameters and operations

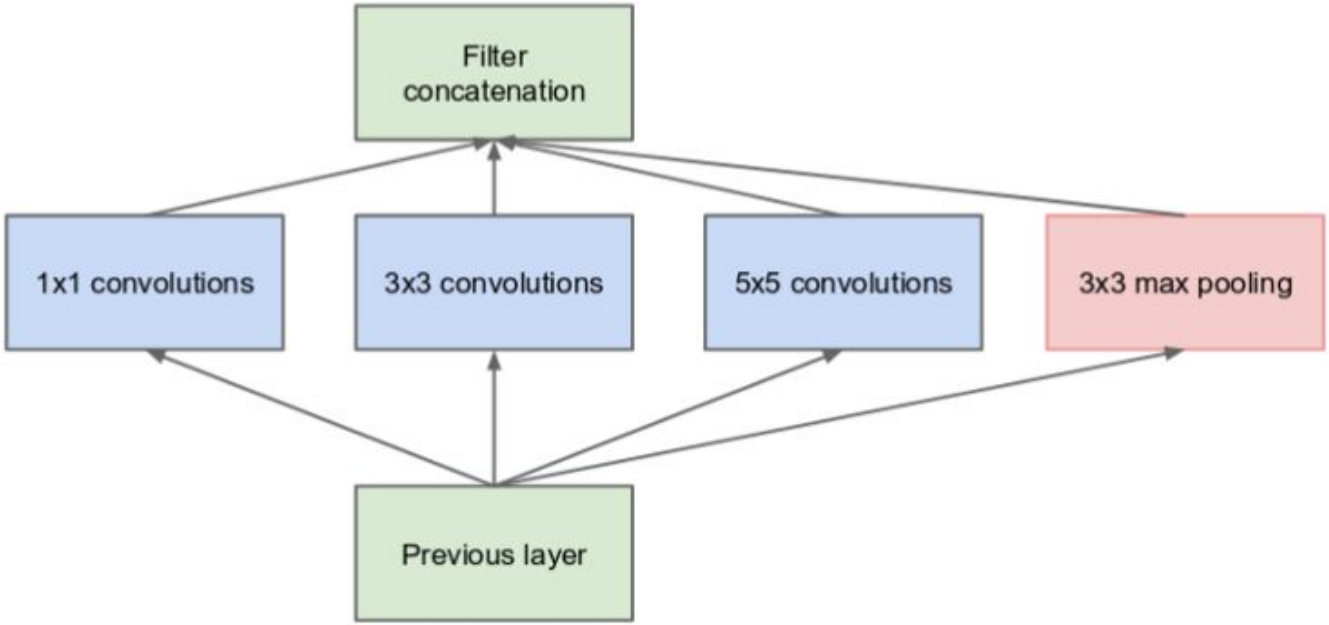
[2] SZEGEDY, C., et al (2014)

Inception: Multiscale

- Objects in an image can be of multiple scales
  - Low scale objects information can be captured by small filters
  - High scale objects need bigger filters (or deeper networks, remember receptive field)

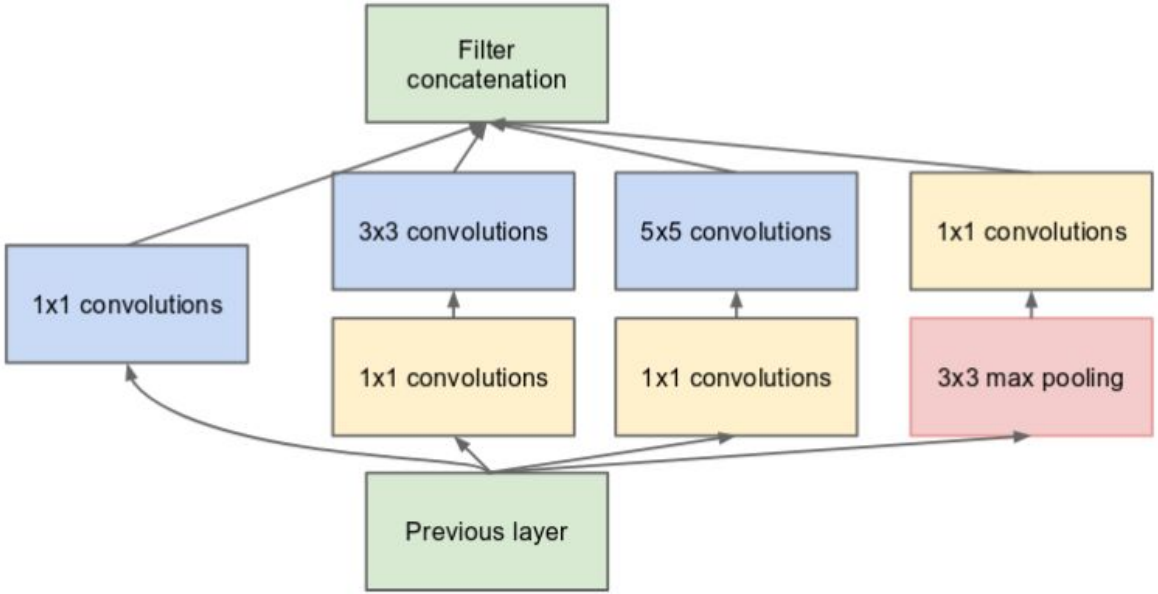
- Idea
  - Have multiple filter sizes at the same layer and concatenate their outputs
  - Let the network learn which filters are more useful

Inception: Inception Module



(a) Inception module, naïve version

Inception: Inception Module (bottleneck)



(b) Inception module with dimension reductions

Inception

Why use 1x1 filters before 3x3 and 5x5 filters?

- Let's compare the number of parameters associated with 5x5 filters of the first inception block
  - Input channels: 192
  - 5x5 filters: 32
  - No 1x1 filters
    - Weights: 192x5x5x32 = 153600
  - 16 1x1 filters
    - Weights: 192x1x1x16 + 16x5x5x32 = 15872
  - That is almost a 90% of reduction of parameters associated with the 5x5 filters of that layer

Inception

What are the 1x1 convolutions doing?

- Projecting the features of the previous layer into a more dense, compressed form
- Same concept as embedding

Inception: GoogLeNet

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

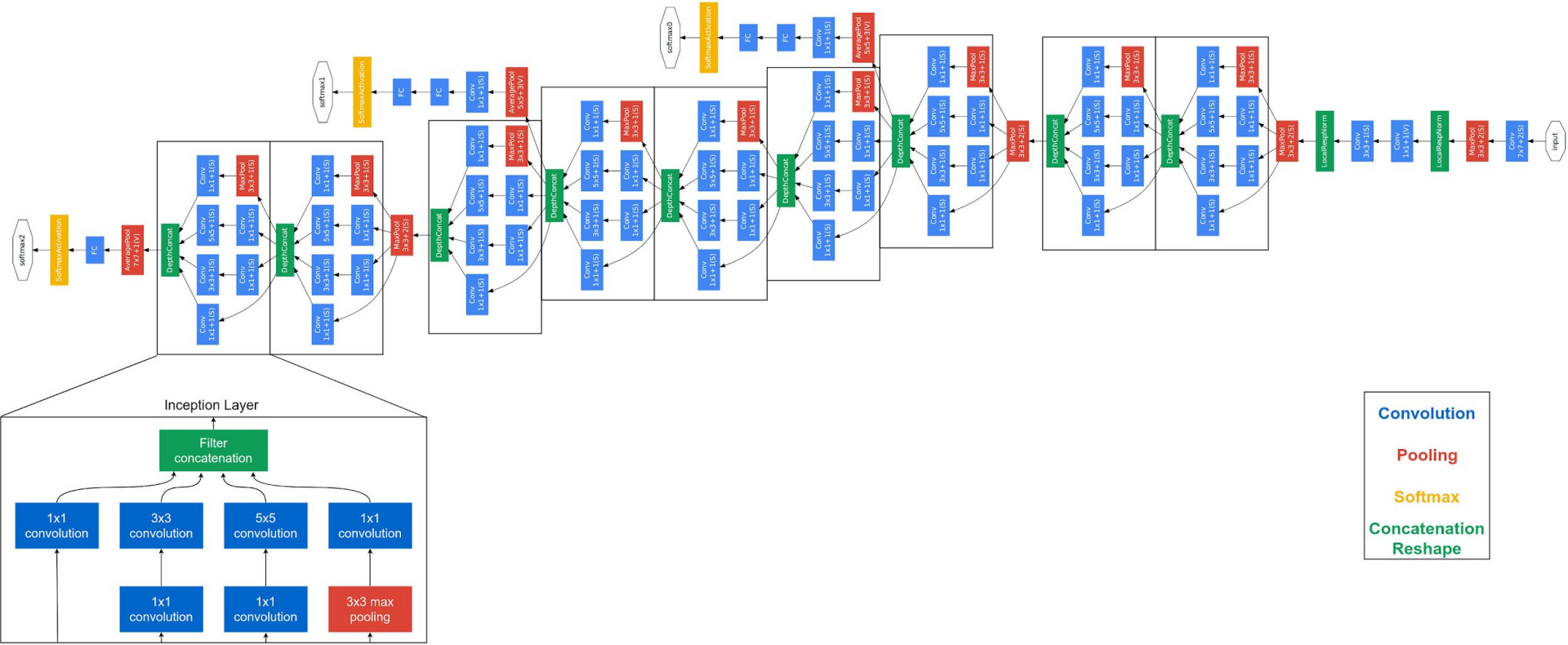
Table 1: GoogLeNet incarnation of the Inception architecture

Inception: Average Pooling

Notice how there is an average pooling layer between the last convolutional layer and the linear layer. This greatly reduces the amount of parameters.

7x7x1024x1000 vs 1x1x1024x1000

Inception


Inception Layer
Filter concatenation
1x1 convolution
3x3 convolution
5x5 convolution
1x1 convolution
1x1 convolution
1x1 convolution
3x3 max pooling
Convolution
Pooling
Softmax
Concatenation
Reshape

Inception

Why are there 3 softmax layers?

- Deeper networks suffer from vanishing gradient
  - 2 auxiliary classifiers were added to intermediate layers to help with the propagation of gradient

Resnet

- A network of N+1 layers should be at least as good as the same network with N layers
    - If the extra layer isn’t useful the network should learn the identity function for that layer

[3] HE, Kaiming, et al.

Resnet: Extra layers, reality

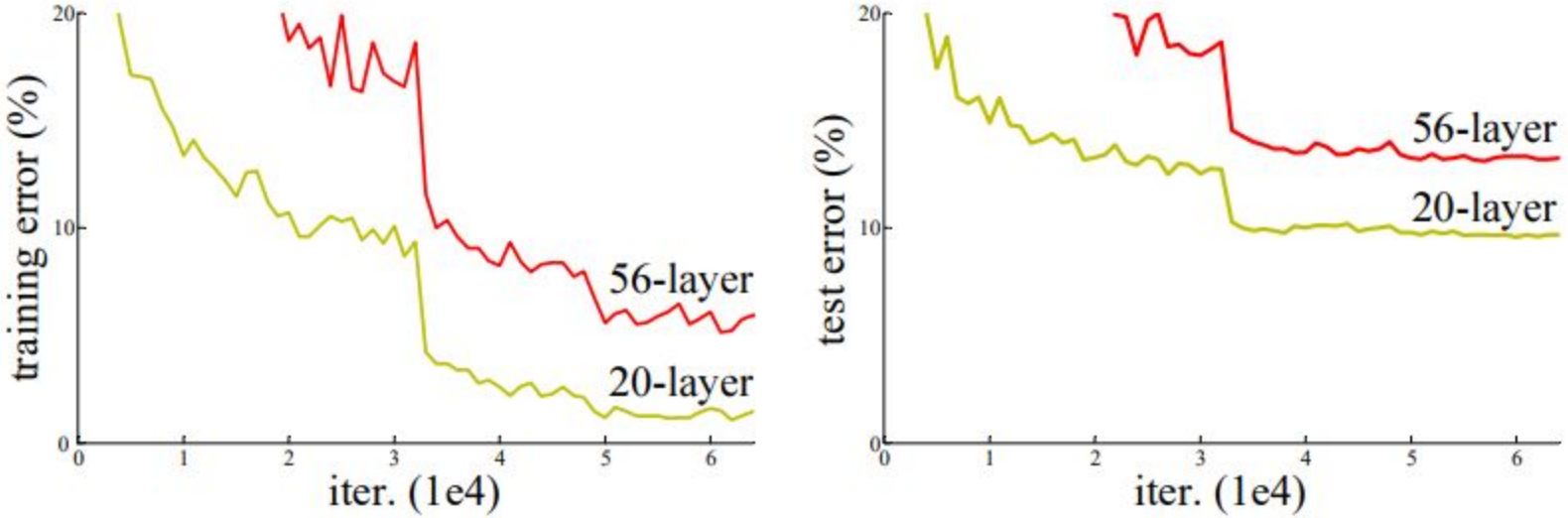


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error.

Resnet

- Hypothesis
  - Not all systems are similarly easy to optimize

- Proposal
  - Let $\mathcal{H}(\mathbf{x})$ be the desired mapping
  - Define the residual mapping $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$.
  - $\mathcal{H}(\mathbf{x})$ becomes $\mathcal{F}(\mathbf{x}) + \mathbf{x}$
  - Learn residuals mappings
    - If identity mapping is the optimal solution is easier to make the weights go to 0

Resnet: residual block

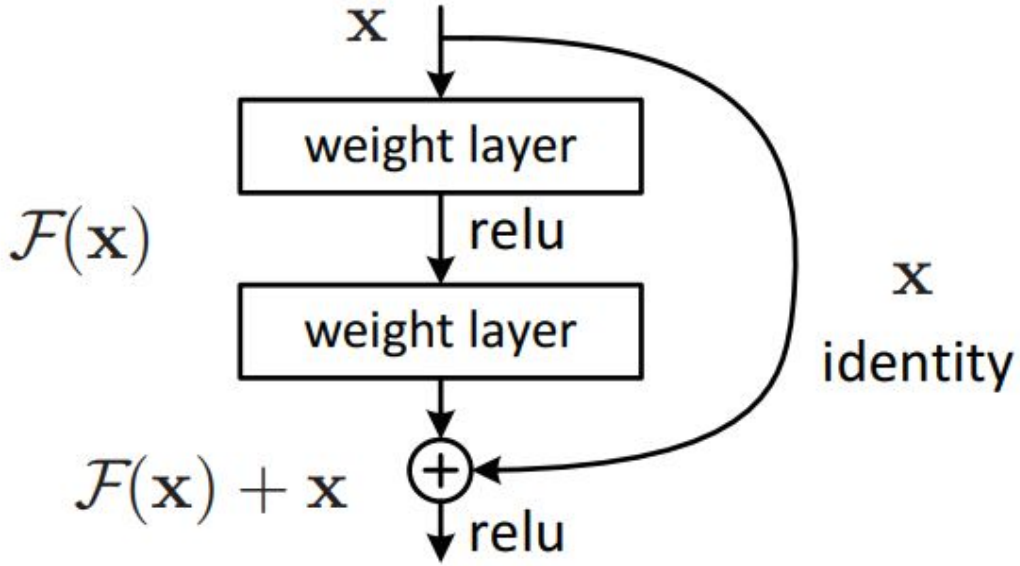
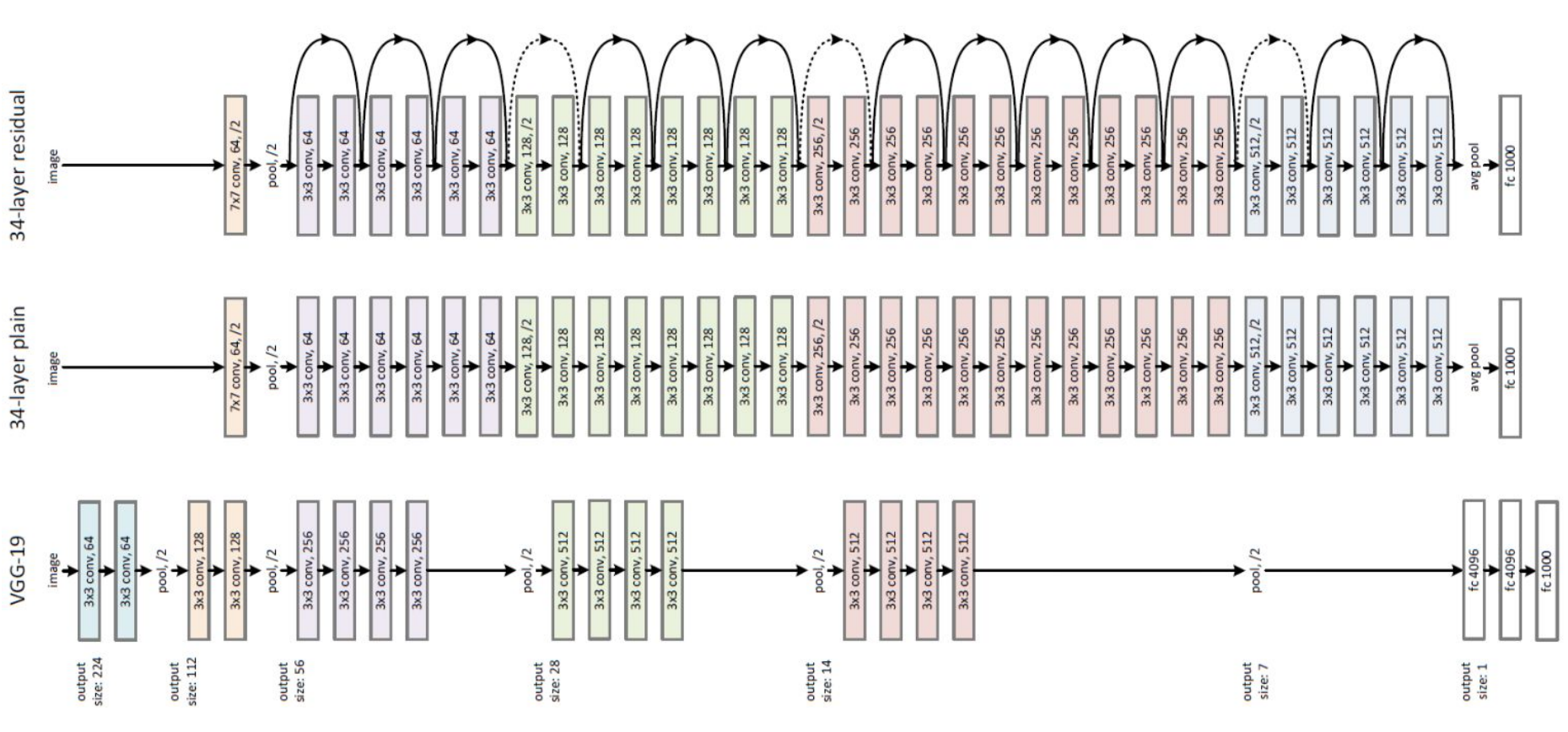


Figure 2. Residual learning: a building block.

Resnet


VGG-19
image
3x3 conv, 64
3x3 conv, 64
pool, /2
output size: 224
3x3 conv, 128
3x3 conv, 128
pool, /2
output size: 112
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
pool, /2
output size: 56
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool, /2
output size: 28
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool, /2
output size: 14
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
pool, /2
output size: 7
fc 4096
fc 4096
fc 1000
output size: 1
34-layer plain
image
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 256, /2
3x3 conv, 256
3x3 conv, 512, /2
3x3 conv, 512
avg pool
fc 1000
34-layer residual
image
7x7 conv, 64, /2
pool, /2
3x3 conv, 64
3x3 conv, 128, /2
3x3 conv, 128
3x3 conv, 256, /2
3x3 conv, 256
3x3 conv, 512, /2
3x3 conv, 512
avg pool
fc 1000

Resnet: Bottleneck block

Resnet applies the same idea of bottleneck to reduce parameters

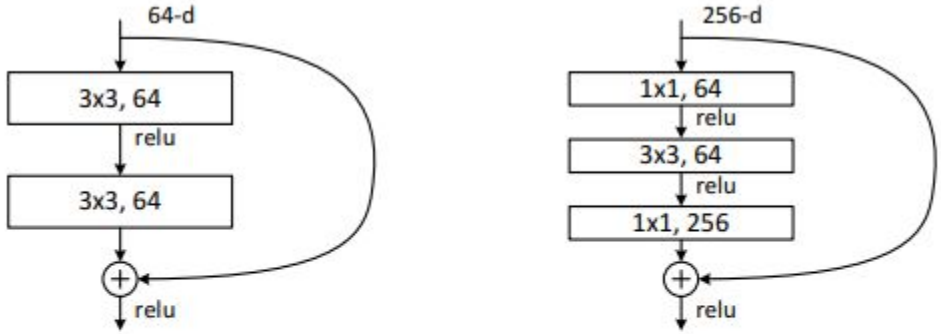


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a "bottleneck" building block for ResNet-50/101/152.

Resnet: Configurations

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Table 1. Architectures for ImageNet. Building blocks are shown in brackets (see also Fig. 5), with the numbers of blocks stacked. Down-sampling is performed by conv3_1, conv4_1, and conv5_1 with a stride of 2.

Resnet: Other decisions

- Batch Normalization after all convolutional layers

- No dropout

Resnet: Results

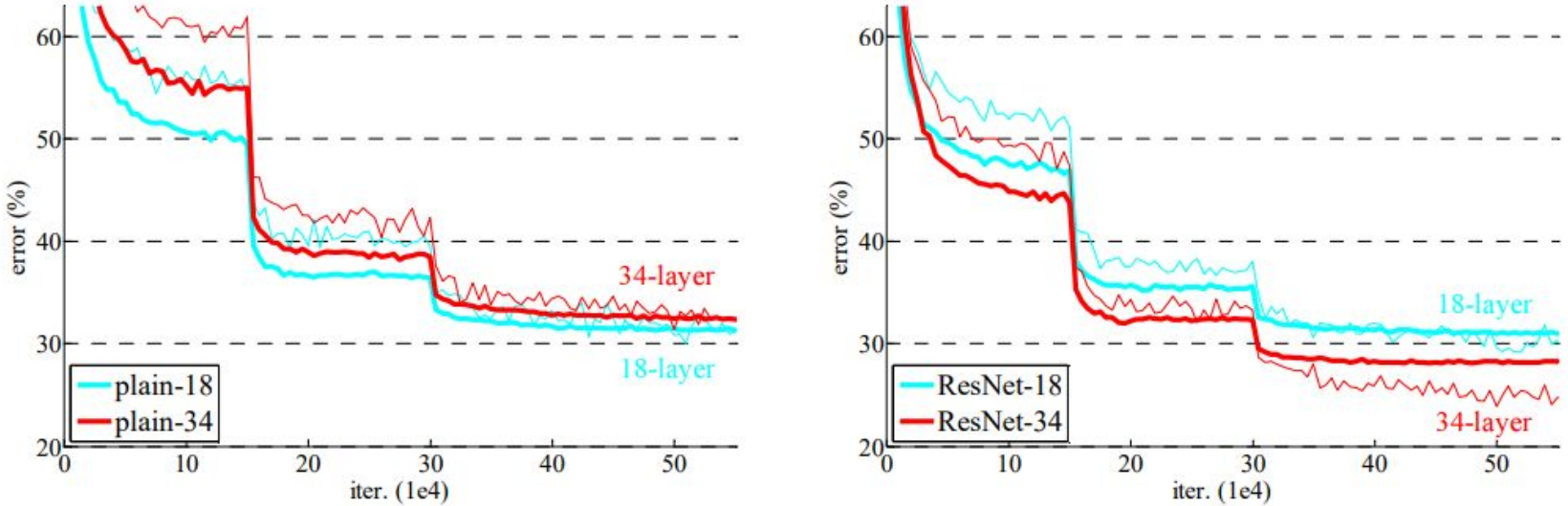
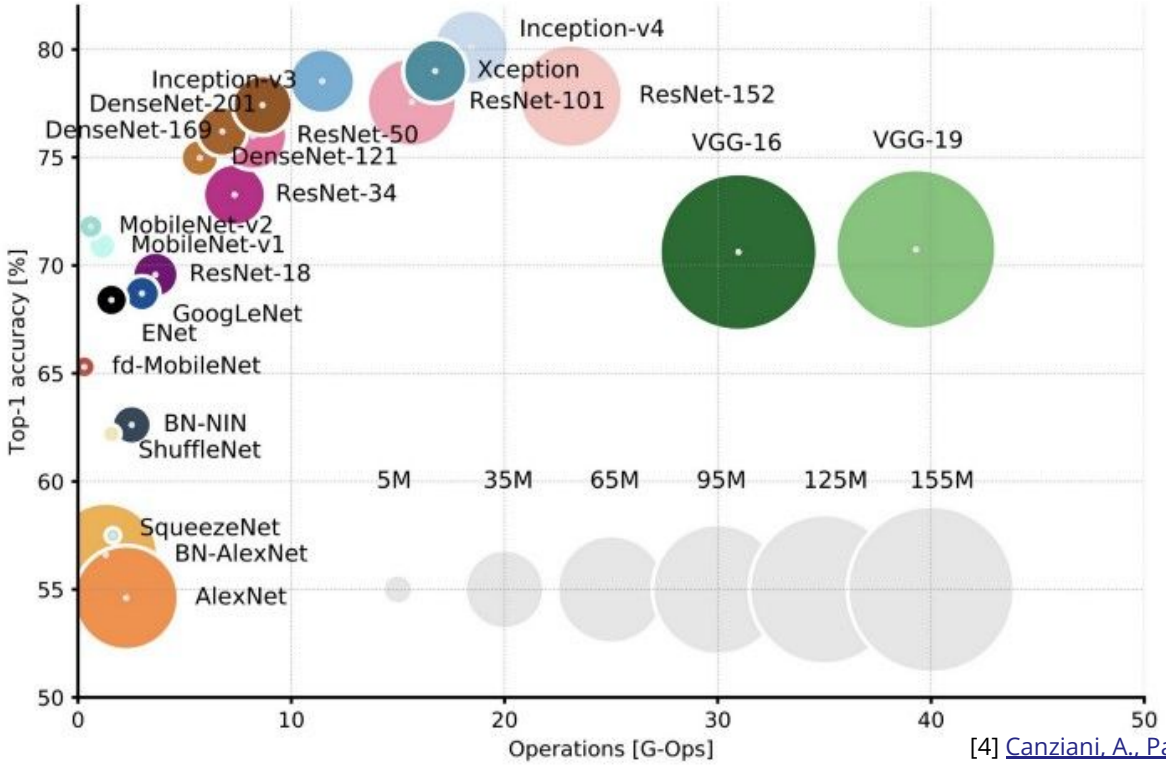


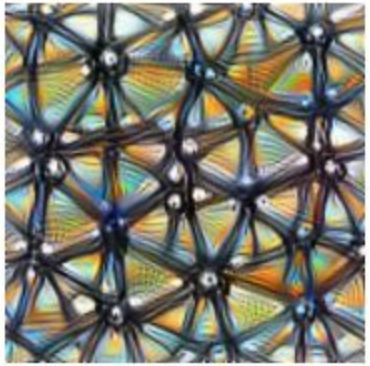
Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

Deep Learning Architectures

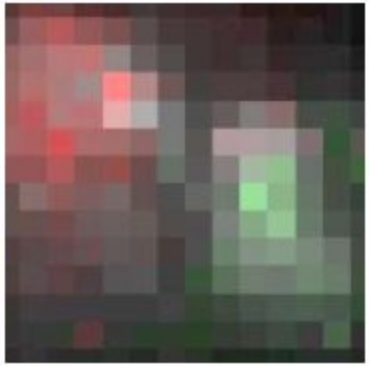


[4] Canziani, A., Paszke, A., & Culurciello, E. (2016)

Interpretability



Feature visualization answers questions about what a network — or parts of a network — are looking for by generating examples.

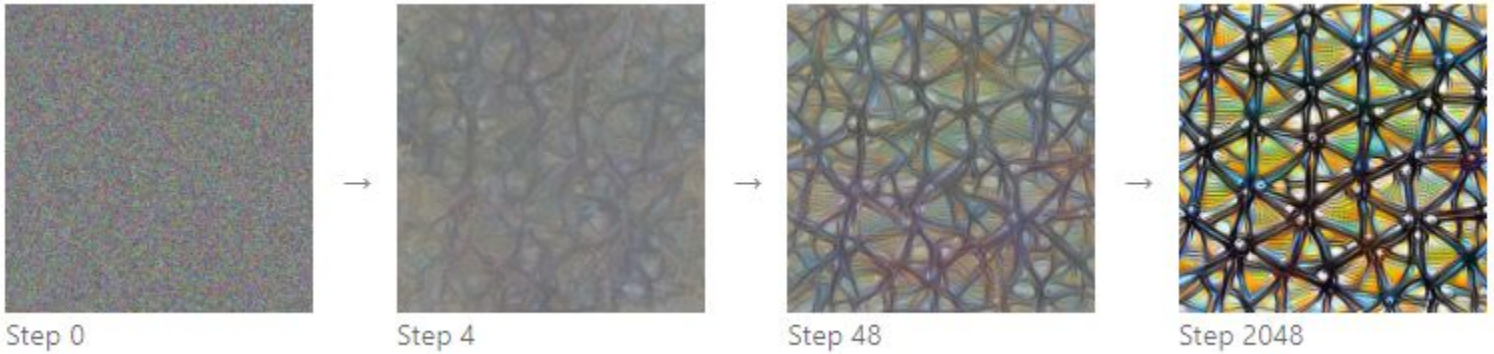


Attribution [1] studies what part of an example is responsible for the network activating a particular way.

Feature Visualization

Networks are differentiable with respect to their inputs.

- We can generate an image that maximizes the desired part of the network



Optimization starting from random noise in search of an image that activates the channel 11 of layer mixed4a of GoogLeNet

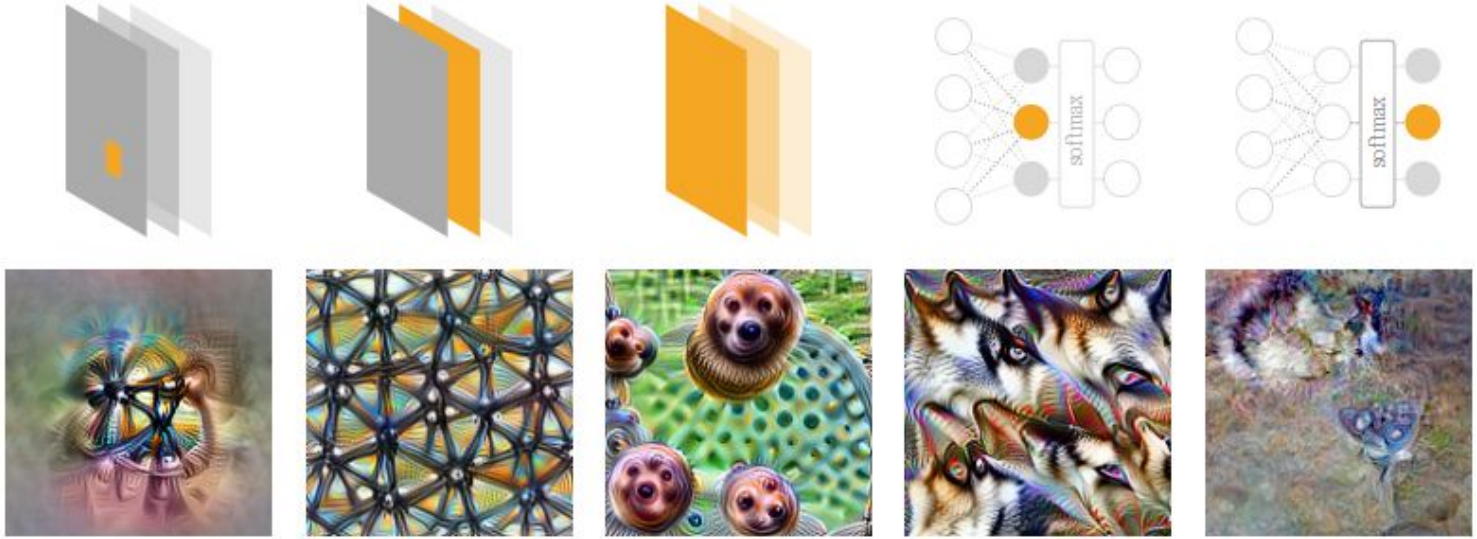
[5] Olah, et al., "Feature Visualization", Distill, 2017

Feature Visualization: Objectives

We can search for several input types

Different **optimization objectives** show what different parts of a network are looking for.

- `n` layer index
- `x,y` spatial position
- `z` channel index
- `k` class index



Neuron
`layer_n[x,y,z]`

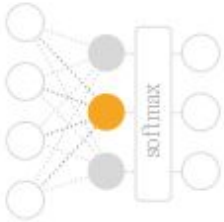
Channel
`layer_n[:,:,z]`

Layer/DeepDream
`layer_n[:,:,:]^2`

Class Logits
`pre_softmax[k]`

Class Probability
`softmax[k]`

Feature Visualization



Class Logits

`pre_softmax[k]`

Class Probability

`softmax[k]`

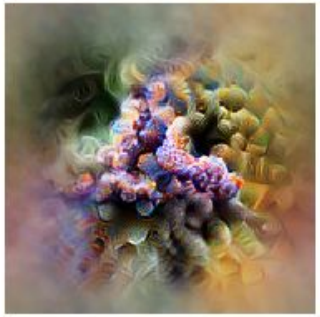
- Both logits and the class probability can be used to obtain an example of the output classes.
- Optimizing logits produces images with better visual quality

Feature Visualization

Dataset examples vs optimization

Dataset Examples show us what neurons respond to in practice

Optimization isolates the causes of behavior from mere correlations. A neuron may not be detecting what you initially thought.



Baseball—or stripes?
mixed4a, Unit 6

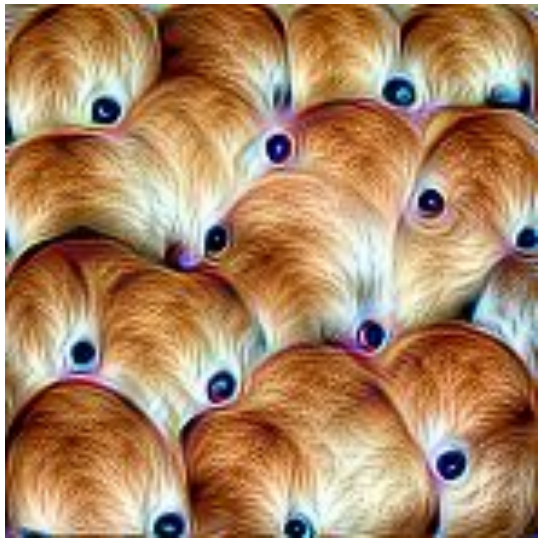
Animal faces—or snouts?
mixed4a, Unit 240

Clouds—or fluffiness?
mixed4a, Unit 453

Buildings—or sky?
mixed4a, Unit 492

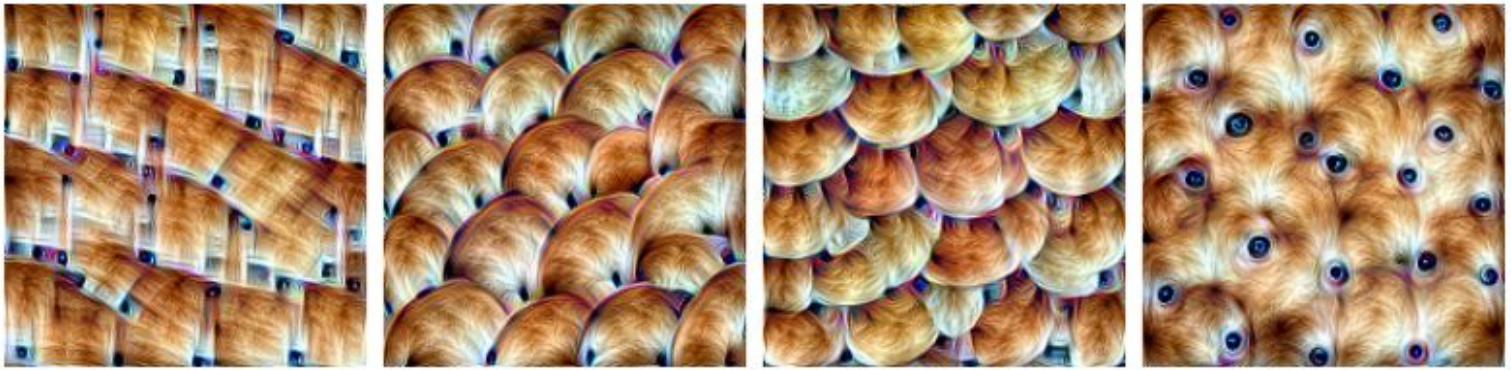
Feature Visualization: Diversity

We have to be careful with considering only 1 image to be representative of what activates our target.



This channel seems to activate with dog heads, but… does it?

Feature Visualization: Diversity

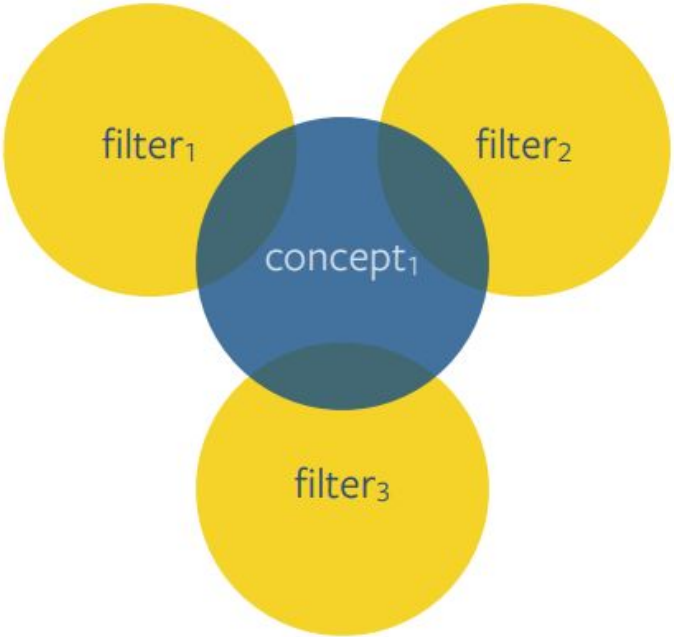


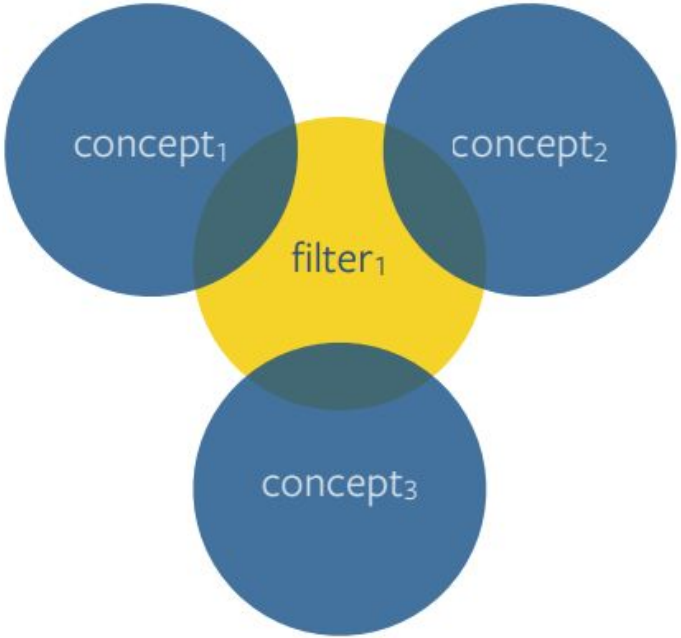
Optimization with diversity. *Layer mixed4a, Unit 143*

Dataset examples

Diversity term into the image optimization reveals different aspects of what really activates the target.

Feature Visualization: Filter-Concept Overlap





Feature Visualization: Filter-Concept Overlap

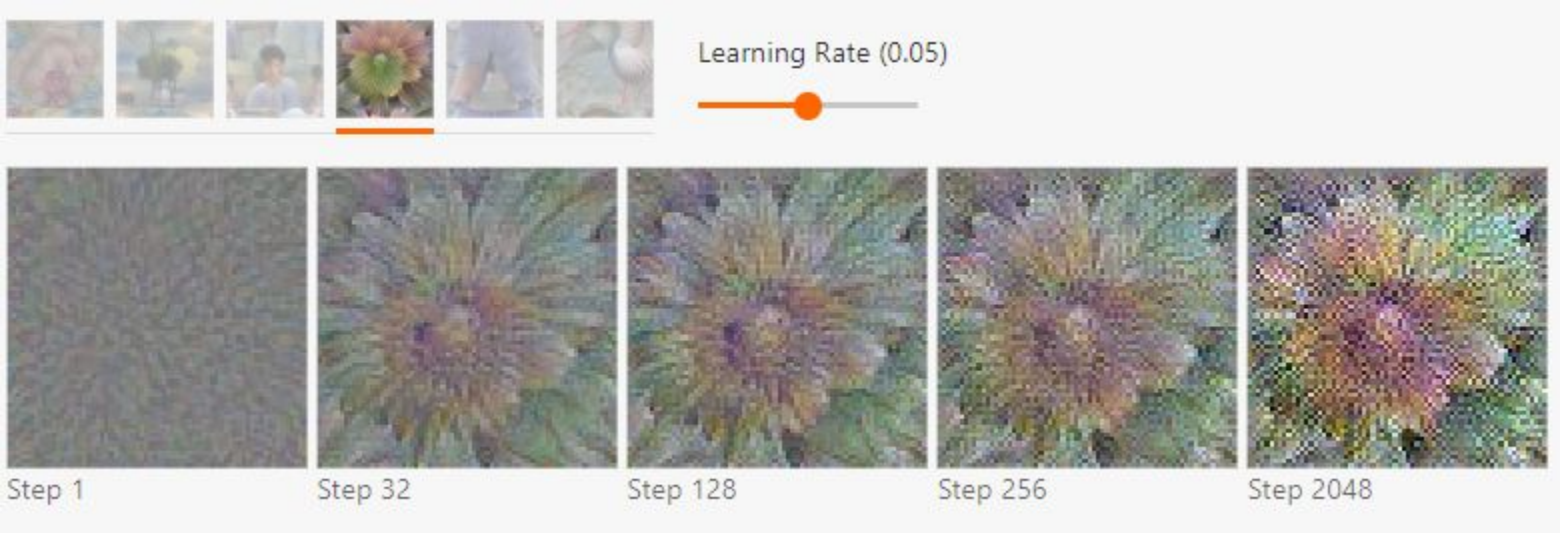
Simple Optimization

Optimization with diversity show cats, foxes, but also cars. *Layer mixed4e, Unit 55*

Dataset examples

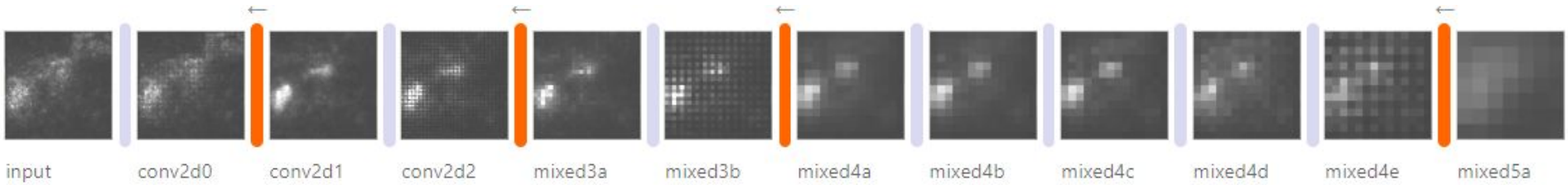
Feature Visualization: reality

Simply optimizing generates noisy images



Feature Visualization

Strided layers create checkerboard patterns in the gradient when backpropagation goes through them



Feature Visualization

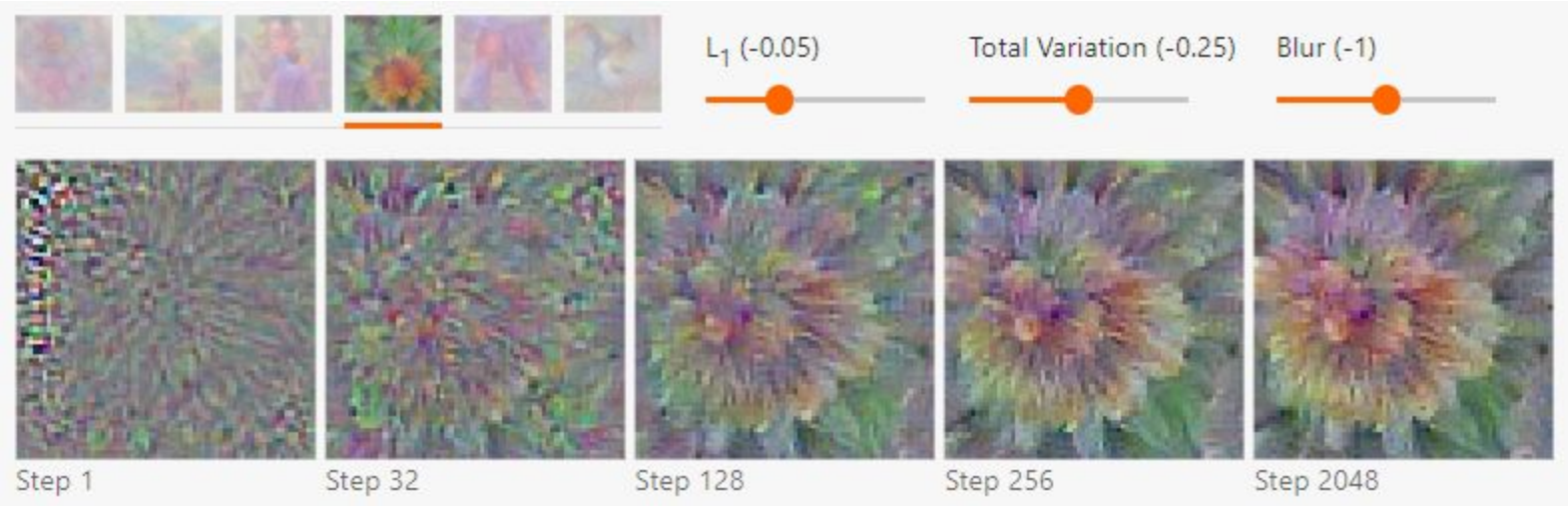
Unrestricted image optimization generates adversarial examples

We can add the following regularization to generate more natural images

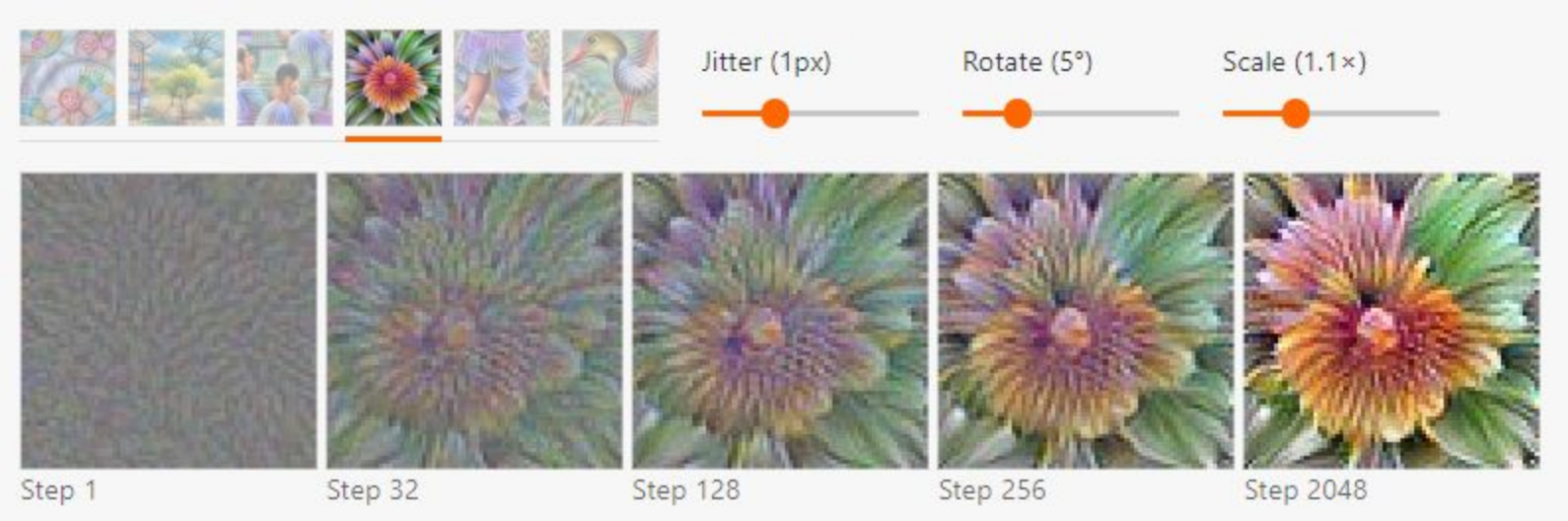
- Frequency penalization

- Transformation robustness

Feature Visualization: Frequency penalization

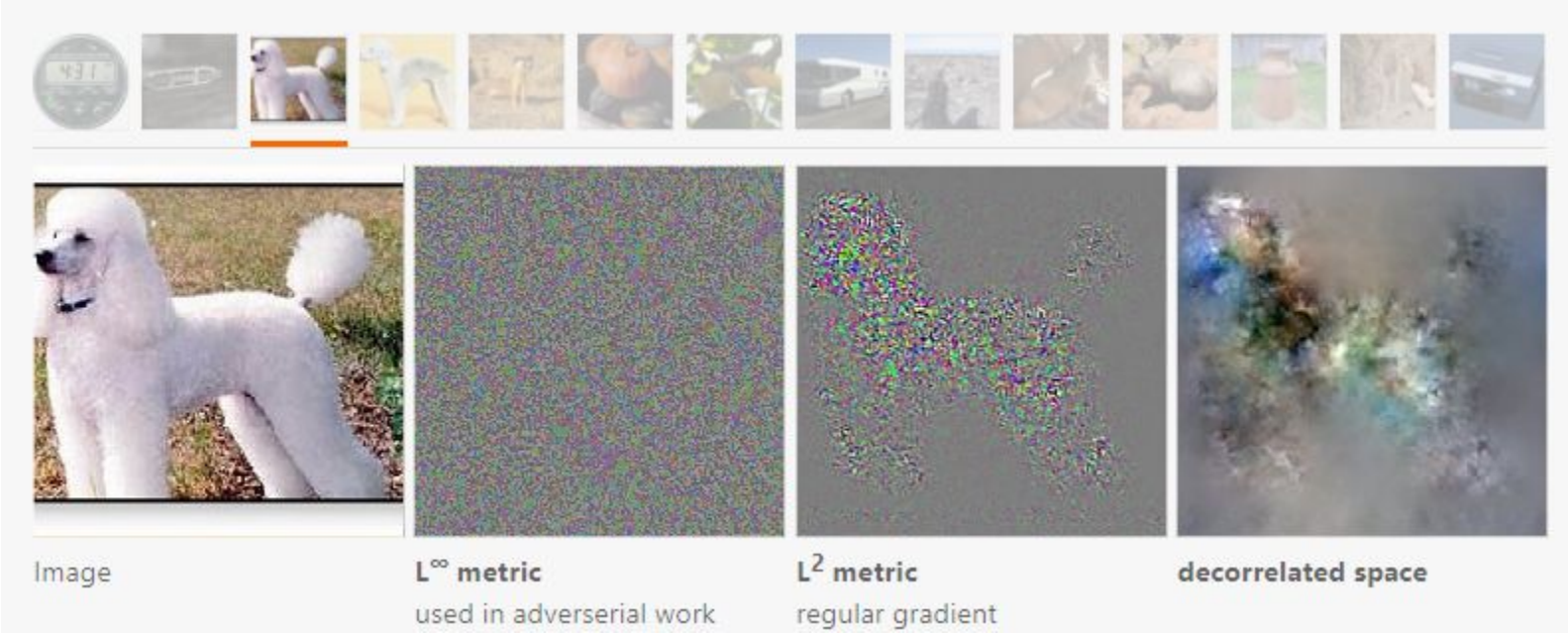


Feature Visualization: Transformation robustness

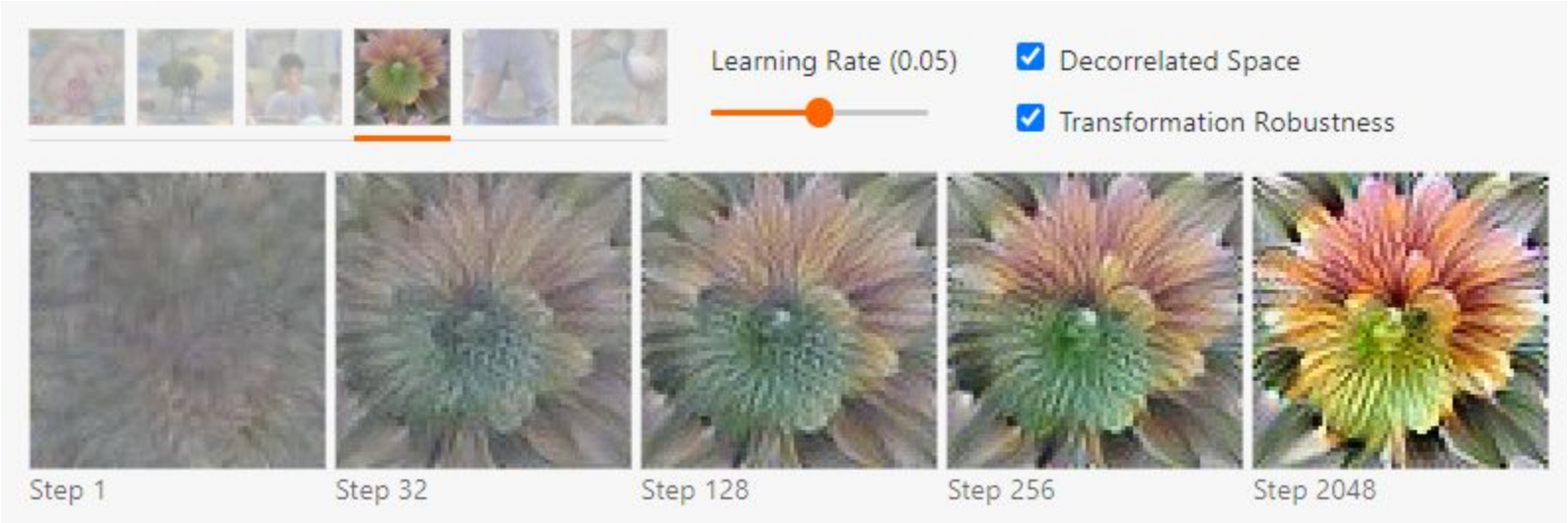


Feature Visualization: Natural images

Changing the metric distance and the input parameterization helps too



Feature Visualization: Final results



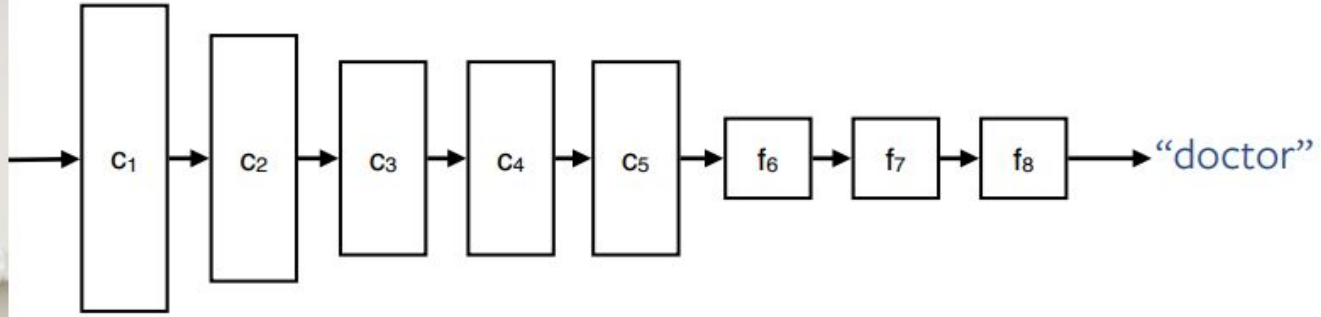
Feature Visualization

GoogLeNet (Resnet v1) trained on ImageNet

Edges (layer conv2d0) **Textures** (layer mixed3a) **Patterns** (layer mixed4a) **Parts** (layers mixed4b & mixed4c) **Objects** (layers mixed4d & mixed4e)

Attribution

Determine which part of the input is responsible for the output



?

Attribution: Why?

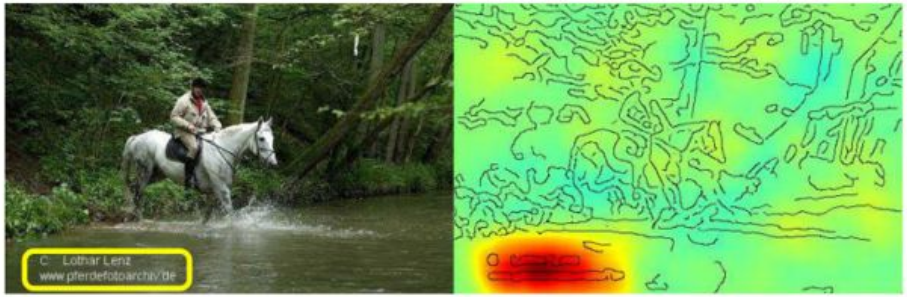
Verify that

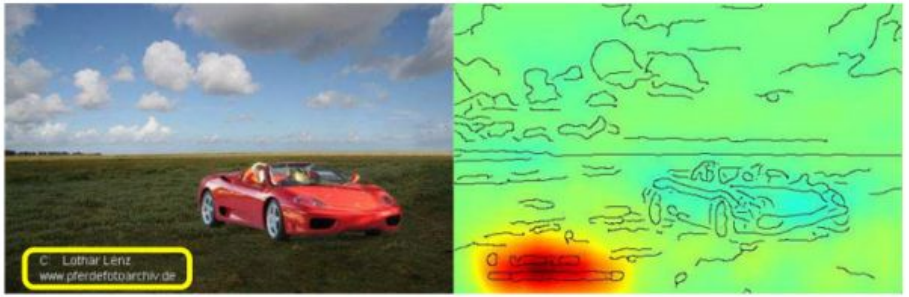
- No wrong biases

- No harmful shortcuts

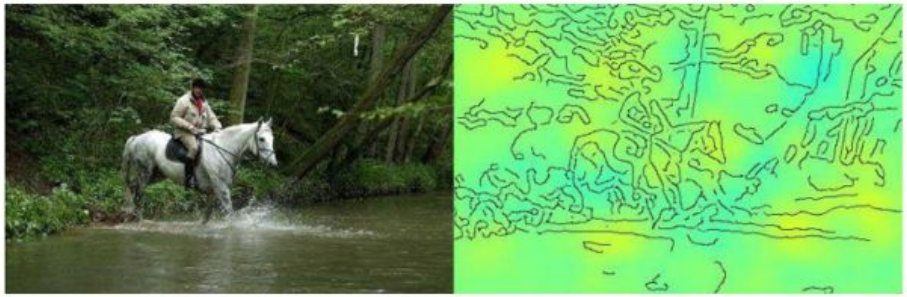
Attribution: Debugging

"horse"





"not horse"



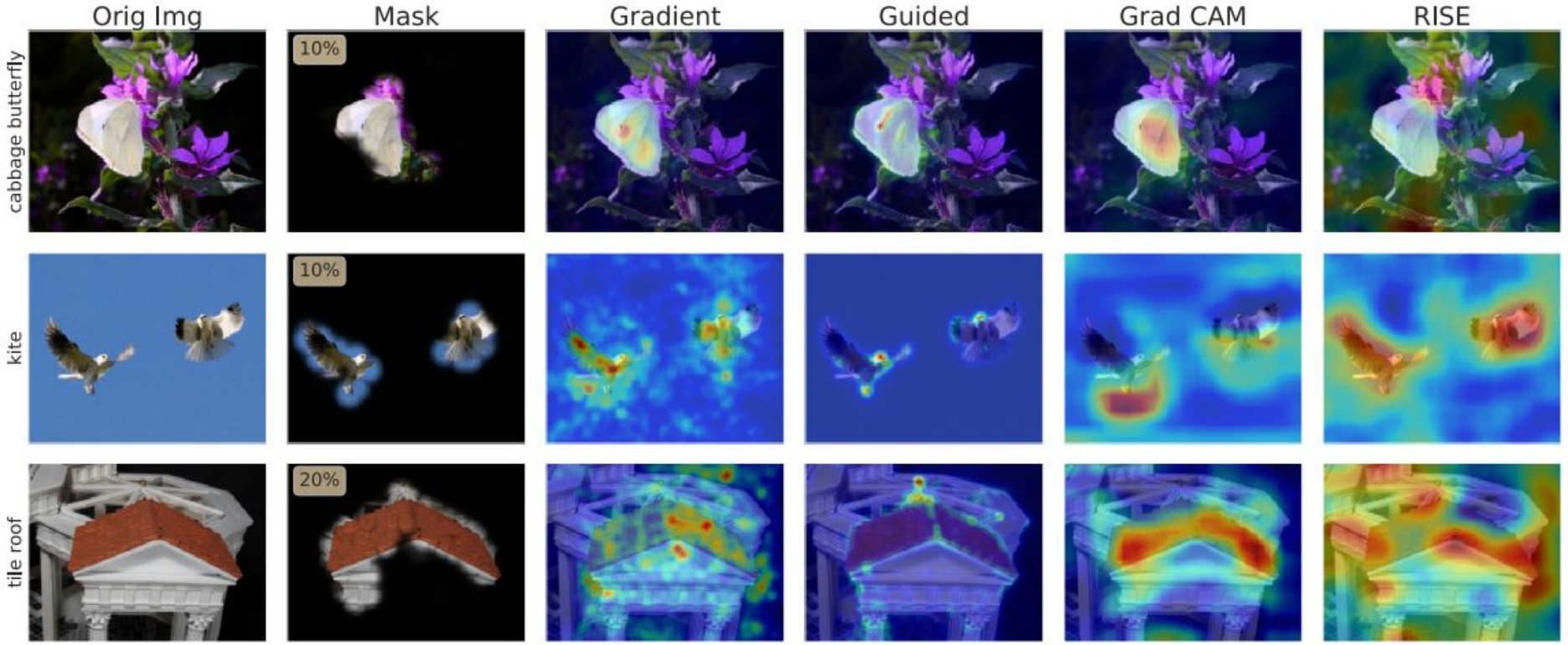
Attribution: Techniques

There are 2 principal branches of attribution methods

- Backpropagation
  - Modify the backpropagation rules of the layers to generate a visualization of the network forward pass

- Perturbation
  - Change the input in a controlled manner and observe the outcome

Attribution: Comparison

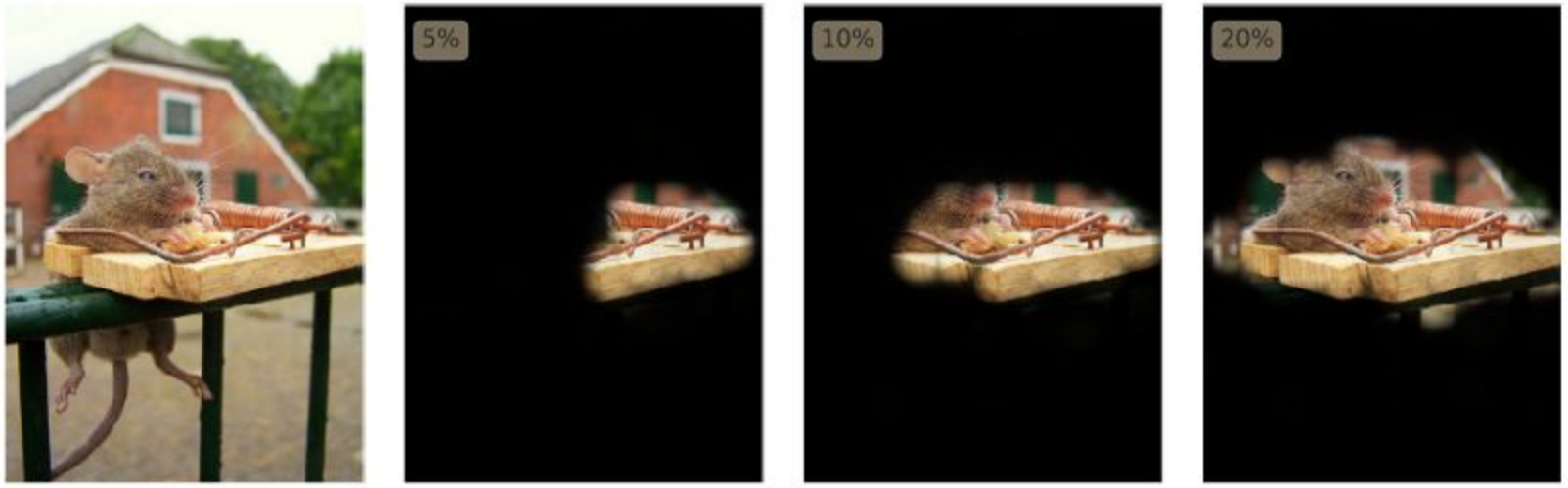


Attribution

We’ll focus on perturbation methods, particularly on Extremal Perturbation (EP) as it generates the best visualizations so far.

Extremal Perturbation

Learn a **fixed-size** mask **m** to perturb input **x** that maximally preserves the network's output



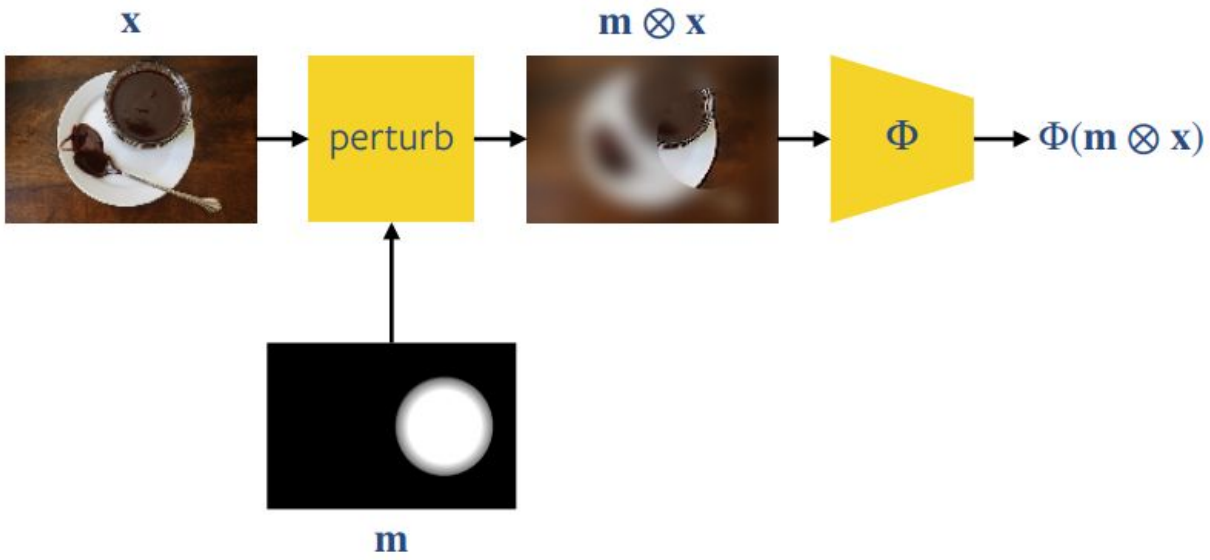
[6] Fong, R., Patrick, M., & Vedaldi, A. (2019)

Extremal Perturbation

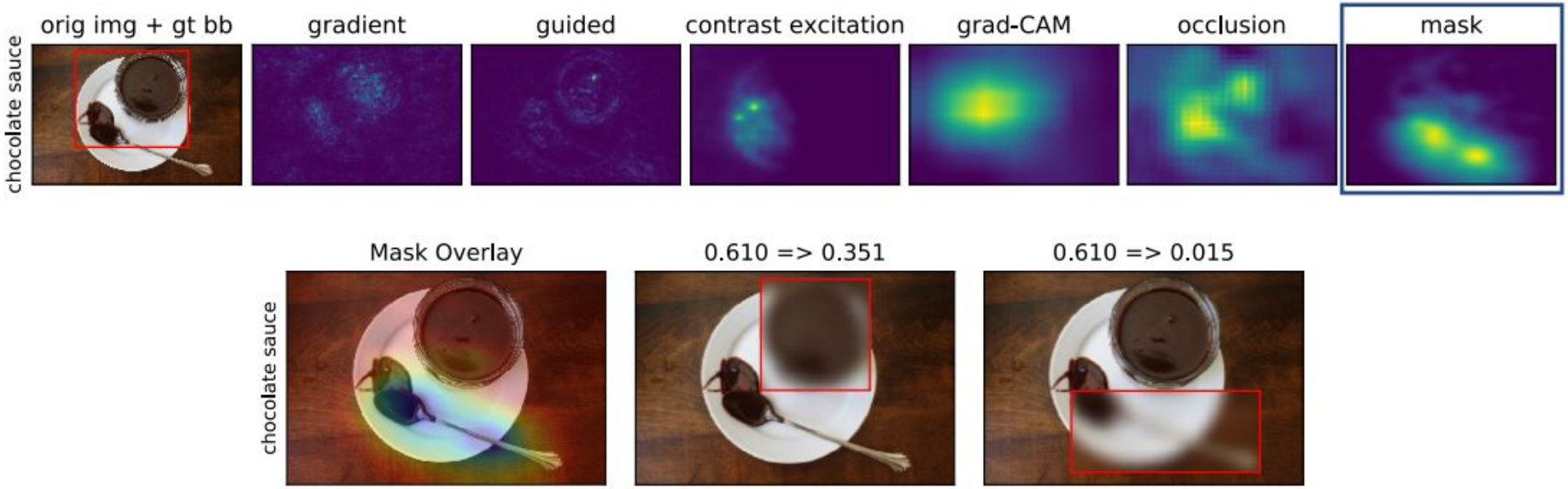
A mask is optimized to maximally excite the network:

$$\underset{\mathbf{m}}{\operatorname{argmax}}\ \Phi(\mathbf{m} \otimes \mathbf{x})$$

subject to $\text{area}(\mathbf{m}) = a$

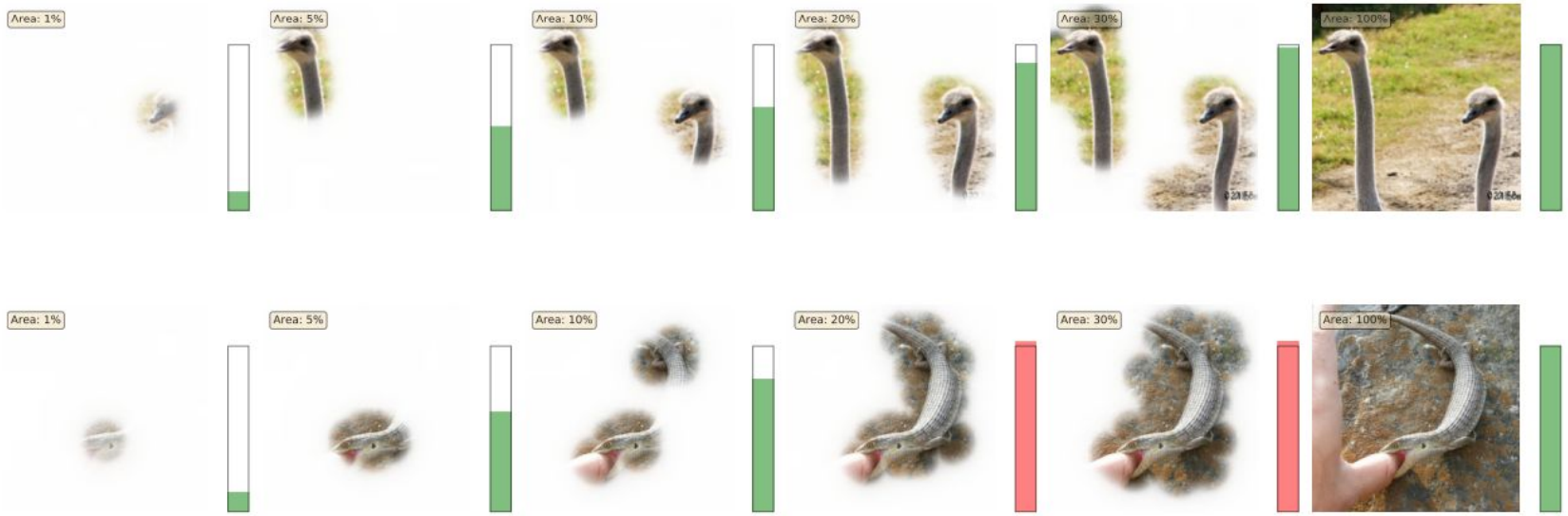


Extremal Perturbation



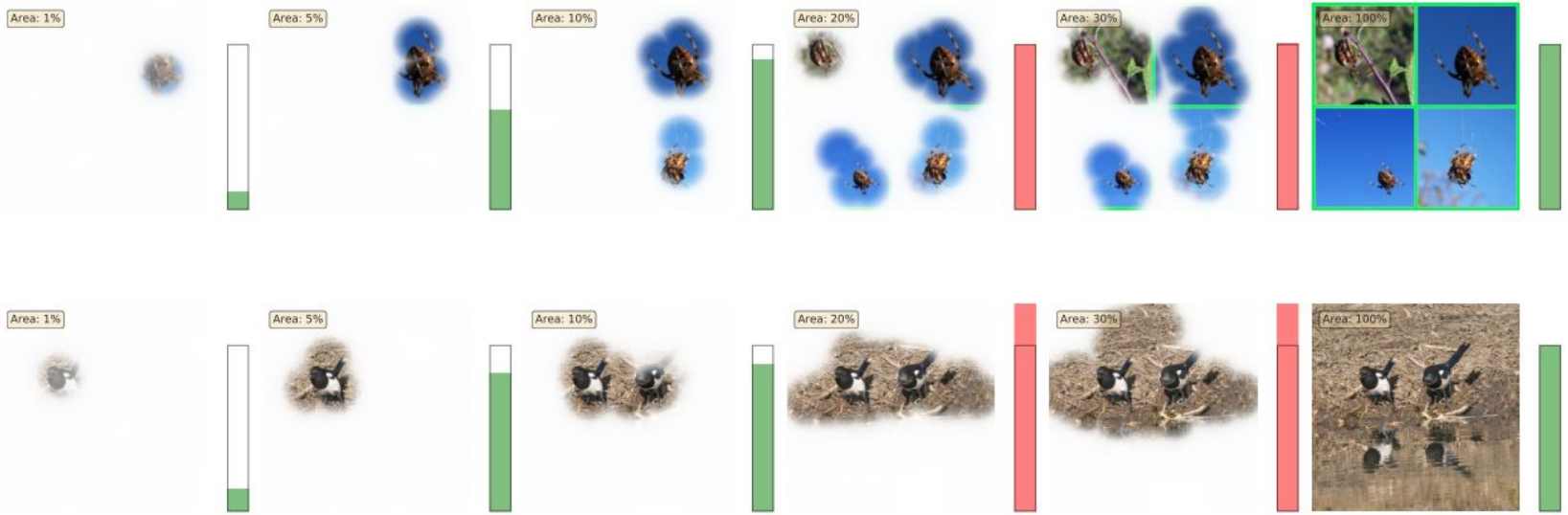
Extremal Perturbation: Insights

Foreground evidence is sufficient



Extremal Perturbation: Insights

Multiple objects contribute cumulatively



References

[1] SIMONYAN, Karen; ZISSERMAN, Andrew. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556, 2014.

[2] SZEGEDY, C., et al. Going deeper with convolutions. arXiv 2014. arXiv preprint arXiv:1409.4842, 2014, vol. 1409.

[3] HE, Kaiming, et al. Deep residual learning for image recognition. En Proceedings of the IEEE conference on computer vision and pattern recognition. 2016. p. 770-778.

[4] CANZIANI, Alfredo; PASZKE, Adam; CULURCIELLO, Eugenio. An analysis of deep neural network models for practical applications. arXiv preprint arXiv:1605.07678, 2016.

[5] OLAH, et al., "Feature Visualization", Distill, 2017.

[6] FONG, Ruth; PATRICK, Mandela; VEDALDI, Andrea. Understanding deep networks via extremal perturbations and smooth masks. En Proceedings of the IEEE International Conference on Computer Vision. 2019. p. 2950-2958.